Topic-1, Python

Datatypes : int, float, str, bool, list, tuple, set, dict, None

Type	Mutable	Ordered
- list	✓	✓
- tuple	x	✓
- set	✓	x
- dict	✓	✓
- str	x	✓

```
x = [1, 2, 3]
y = x
```
x & y both point to same obj

y = x.copy() want separate list

For nested objs,
```
import copy
y = copy.deepcopy(x)
```

Data structures

```
arr = [10, 20, 30]
arr.append(40)
    .pop()
    .insert(1,15)
    .remove(20)
```

Complexity

Access arr[i]	— O(1)
Append	— O(1) avg
Search	— O(n)
Insert beginning	— O(n)
Delete "	— O(n)

Dictionary

```
student = {"name": "Shiwani", "cgpa": 9.5}   print(student["name"])
```
Hash-table based.
Avg: Insert → O(1) ; Search → O(1) ; Delete → O(1)

```
freq = {}
for x in arr:
    if x in freq:
        freq[x] += 1
    else:
        freq[x] = 1
```

— everywhere in DSA
— Backend Development.

Set : Use when you mainly care about uniqueness / fast membership checking.

— Average lookup : O(1)
— classical use :
```
arr = [1,2,2,3,3,4]
unique = set(arr)
print(unique)
```

```
visited = set()
visited.add(10)
if 10 in visited:
    print("Already visited").
```

Stack and Queue

Stack → LIFO

```
stack = []
stack.append(10)      # push → O(1)
      .append(20)     # pop  → O(1)
x = stack.pop()
```

Queue → FIFO
Don't use a list for large queues because pop(0) is O(n):
→ from collections import deque
```
queue = deque()
queue.append(10)        #Both operations
      .append(20)       are O(1).
x = queue.popleft()
```

Collections

```
from collections import counter,
    defaultdict, deque.
```

→
```
from collections import counter
arr = [1,2,2,3,3,3]
freq = counter(arr)
print(freq)   # {3:3, 2:2, 1:1}
```

→ defaultdict

```
from collections import defaultdict
freq = defaultdict(int)
freq[x] += 1
```

```
freq = {}
if x not in freq:
    freq[x] = 0
freq[x] += 1
```

Functions

```
def add(a,b):          [Normal Func]
    return a+b
```

```
def greet(name = "User"):
    return f"Hello {name}"
```

*args : multiple positional arguments

```
def add(*nums):
    return sum(nums)
add(1,2,3,4).
```

**kwargs (multiple keyword arguments)

```
def info(**data):
    print(data)
info(name = "Shiwani", age = 20).
```

commonly used with : sorted(), map(), filter().

Lambda : Small anonymous

eg :

```
students = [("A",90),("B",80),
            ("C",95)]
```

```
students.sort(key = lambda x : x[1])
```

List comprehension

```
squares = [x * x for x in range(10)]
```

```
even = [x for x in arr if
    x%2 == 0].
```

Exception Handling

```
try:
    x = int(input())
except ValueError:
    print("Invalid Input")
```

Full Features :

```
try : #risky code
except ValueError : # handle specific error
else : #runs if no exception
finally : #always run
```

File Handling

```
with open("data.txt","r") as file :
    data = file.read()
```

```
with open("data.txt","w") as file :
    file.write("Hello").
```

Why with ?
- It automatically handles closing the file.
- It automatically handles closing the file.

Modules & Packages

```
import math      or      from math import sqrt.
```

Python Iterator & Generator

↳ worth revising it never you beyond.

```
arr = [1,2,3]
it = iter(arr)
print(next(it))
print(next(it))
```

Generator : def numbers():
yield 1
, 2
, 3

for x in numbers():
print(x)

returns: returns result & exists function
yield: produces values one at a time.
Generators are memory efficient for large data.

Decorators

```
def decorator(func):
    def wrapper():
        print("B")
        func()
        print("A").
    return wrapper.
```

```
@decorator
def hello():
    print("hello").
```

Adding behaviours to a function without changing the function itself.

Context Managers

with — uses the context manager protocol.
methods — __enter__(), __exit()
concept — files, DB connection, locks, resources, API clients.

VE + Packages

python -m venv venv
venv/Scripts/activate
python ↑
pip install requests. (Install)
pip freeze requirements.txt [Save dependencies]
pip install -r requirements.txt [Install "]

API: Application Programming Interface.
A client sends a request: Client → HTTP Req → Server → Processing → HTTP response.

HTTP methods:
GET — Fetch data
POST — create "
PUT — Replace / Update "
PATCH — Partially update
DELETE — delete "

```
import requests
response = requests.get
("https://api.ex.com/u")
print(response.status_code)
print(response.json).
```

HTTP Status Code

200 → ok
201 → Created
204 → No content

400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
409 → Conflict.

500 → Internal Server Error
502 → Bat gateway
503 → service unavailable.

401 → authentication problem.
403 → permission problem.

Testing in Python

Testing verifies behavior correct or not.
Main goals: find bugs, verifies reqs, prevent regressions, improve reliability, ensure changes don't break existing functionality.

Types: **Unit**: small individual unit, usually func or method.
Integration: test whether multiple components work correctly together.
API → service → DB (check creating user through an API actually stores to User correctly in DB).

End to End (E2E) Testing: Tests the complete application flow from the user's perspection.
eg: login → search product → add to cart → checkout → Payment.

Regression Testing: checks the new changes haven't broken previously working functionality.
Add login feature → if ensure → signup, old API, Passw. reset still work

Assert: Py provides assert statement for basic checks.
assert 2+3 == 5. False ⇒ AssertionError.

UnitTest: Py has built-in testing framework ⇒ unittest.

```
import unittest
def add(a,b): return a+b
Class TestAdd (unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2,3),5)

if __name__ == '__main__':
    unittest.main()
```

common assertions
- assertEqual()
- assertNotEqual()
- " True()
- " False()
- IsNone()
- In()
- Raises()

Pytest: popular python testing framework.
Install: pip install pytest
Basic Test: def add(a,b): return a+b
def test_add(): assert add(2,3) == 5.
Run: pytest.

Why pytest? — simple syntax, — powerful fixtures, easy parameterization, good error messages, supports mocking, works well for unit & integration tests.

Fixtures : fixtures provide reusable test setup.

```
import pytest
@pytest.fixture
def user():
    return {"name": "Shiwani", "age": 20}
def test_user(user):
    assert user["name"] == "Shiwani"
```

Here, @pytest.fixture
↓
user()
↓
test user(user)

This fixture prepares the data required by the test.

Why fixtures?
→ Instead of repeatedly writing:
user = {...}
In every test, create it once as a fixture.

Fixture scope : diff. lifetimes. common scopes : functh, class module, package, session.
default : function.

#Meaning the fixture runs separately for each test function.

Eg :
```
@pytest.fixture(scope="module")
def database():
```
the fixture can then be shared across tests in the module.

Setup & Teardown : sometimes you need to prepare something before testing & clean it afterward.

Setup → Test → Clean up
(before yield) (after yield)

Parameterization : multiple inputs tested in one test func.

Mocking : means replacing a real dependency with a fake object during testing.

Eg : your application → External API.
calling a real external API during test can be :
slow, expensive, unreliable, dependent on internet availability.
Instead : your applicatn → Mock API. (gives controlled data).

from unittest.mock import mock.

Patch: temporarily replaces an obj/function during test.

Eg: Real functa → temporarily replaced → Mock function → test → original restored.

from unittest.mock import patch.

Useful for: Ext APIs, Email services, Payment services, AI/LLM APIs, File Systems, Time-dependent code.

Mocking in API: Testing the real API isn't ideal. You can mock the request. the test doesn't depend on the real server.

API Testing: For an API, test: Request → Method, URL, Headers, Parameters, Body.

Response: Status Code, Response Body, Headers, Error Msg.

What should API tests check? For a POST /users endpoint:

Valid Request → 201 created.
Invalid req: Missing name → 400 Bad Request
Unauthorized: No authentication → 401 Unauthorized
Forbidden: Insufficient Permission → 403 Forbidden.
Not Found: Unknown User → 404 Not Found.

Unit v/s Integration Testing

Test one component → No real DB, No real API, No ext. dependency.
usually one use mocks.

Integration → API → service → DB (real components interact).

Features	Unit Test	Integration Test
Scope	Small	Multiple components
DB	Usually mocked	Often real / test DB
Ext. API		may be tested
Speed	Fast	slower
Purpose	Logic	Component interacts.

Cold Coverage: measures how much of your code is executed by tests.

Install: pip install pytest-cov
Run: pytest --cov

eg:

Name	Stmts	Miss	Cover
Calculator.py	10	1	90%

90% coverage does not mean 90% bug-free.
→ coverage only tells you which code was executed, not whether your tests were good.

Test - Driven Development : means writing tests before implementing the feature.

Cycle : RED → Write failing test GREEN → Write code to pass
REFACTOR → Improve code.

So, Test → Code → Refactor [called Red - Green - Refactor cycle].

Good Testing Practices

i) Test one behaviour at a time : def test_user_login():...

ii) Keep tests independent : one test shouldn't depend on another test running first.

iii) Test edge cases : empty, zero, Negative, large, Invalid, Boundary.

iv) Use meaningful names : Bad → test_1() ; good = test_login_fails_with_invalid_pswd()

v) Mock external dependencies -
Don't unnecessary call → Real API, Payment gateway, email service. during unit tests.

Important

Test case → Specific scenario being tested.
Test Suite → Collects of tests.
Assertion → Expected Condition
Fixture → Test Setup / reusable data.
Mock → Fake dependency
Stub → Provides predefined responses.
Regression → Previously working feature breaks
Coverage → Code executed by tests.
Unit Test → Tests individual component.
Integration Test → Tests component interaction.
E2E Test → Test complete user flow.

#Shiwanish

Python APIs

API → Application Programming Interface.
one S/W system to communicate with another S/W system

For example: your Python App → API req → Weather server →
API Response → your python App.

→ Py program doesn't need to know how the weather service internally works. It simply follows API's rules.

Simple example: Want weather of current.
your Py program sends: GET /weather?city=Bhopal
The server might return: {city=Bhopal, temperature=28, condition=Cloudy}
→ your program can then use that data.

Why do we need APIs?

→ Without APIs, applications would need direct access to the internal implementation of other systems.
→ APIs provide a controlled interface.
→ eg: Frontend → Backend API
Mobile App → Backend API
Python script → Google API
your Application → Gemini API
Website → Payment API
Application → DB service.

[An API acts as a contact/interface between systems.]

Client and Server

↳ Most web APIs follow a client-server architecture.
↳ Client: the system making the request. eg: Browser, React Application, Mobile App, Python Program.
↳ Server: the system receiving the request & processing it.
client → Request → server → Response → client.

(#) Python Program → api.example.com → DB → server processes request → JSON response.

(#) API v/s Database: These are not the same thing.
A database stores data. An API provides controlled access to functionality/data. [Frontend → API → Backend → DB]

The frontend generally shouldn't directly connect to DB
The backend / API handles — Validation, Authentication, Authorization
business logic, DB operations.

Web APIs: When people talk about APIs in Web Dev, APIs that communicate over HTTP/HTTPS → Client sends HTTP req
Eg: https://api.example.com/users → Server sends an HTTP response

HTTP: Hyper Text Transfer Protocol.
→ Rules clients & servers communicate over the web
HTTPS is HTTP over an encrypted connection using TLS.
Client → HTTP req → Server → HTTP response → Client

Anatomy of URL https://api.example.com/users/25?active=true
Protocol | host / Domain | path | Query Parameter.

EndPoint: Specific API URL through which a particular resource or functionality can be accessed.
Eg: GET/users GET/users/25 POST/users
There can be different operations involving the same resource.

HTTP Methods
GET → Retrieve data [GET/users.
POST → Create data / Perform an action [POST/users
PUT → Replace an existing resource
PATCH → Partially update a "
DELETE → Delete a resource. DELETE/users/10

CRUD: Create, Read, Update, Delete.

Typical API:
POST/users → Create
GET " → Read all
GET/users/10 → read one
PUT ——— → Update
PATCH ——— → Partial Update
DELETE ——— → Delete

HTTP Request: contain Method, URL, Headers, Query Parameters, Path Parameters, Body

```
POST/users?notify=true HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer TOKEN
{"name": "Shivani"}
```

HTTP Response : A response generally contains : Status Code, Headers, Body.

Eg : HTTP/1.1 201 Created
Content-Type : Application/json
{ "id" : 101 , "name" : "shiwani" }

HTTP Status Code

200 - OK (GET, PUT, PATCH)
201 - Created (POST)
204 - No content (DELETE)
500 - Internal server Error
502 - Bad Gateway
503 - service Unavailable

400 - Bad Request (req invalid)
401 - Unauthorized (No valid token).
403 - Forbidden (not authorized for this)
404 - Not Found (doesn't exist).
409 - Conflict (Creating username already exists)
429 - Too many Req. (rate limit exceed)

Path Parameters : identify specific resource.
GET /users/25, user id

Query Parameter : Used to provide optional filtering, Sorting, searching, pagination, etc.

Difference :
/users/25 → Path parameter identifies a particular resource.
/users ?city=Bhopal → Query parameter modifies the request.

Request Body : Body contains data sent to the server, commonly with POST/PATCH/PUT.
→ usually sent as JSON in REST APIs.

JSON : Javascript Object Notation. [common format for API communication]
{ "name" : "shiwani" , "age" : 20 , "skills" : ["Python" , "Django"] }

Important :

JSON obj	→	Python	dict
" array	→	"	list
" String	→	"	str
" number	→	"	int/float
" true	→	"	True
" false	→	"	False
" null	→	"	None

Python requests : The most common beginner friendly library for making HTTP requests.
Install : pip install requests
Import : import requests.

GET Request in Python

```
import requests
url = "https://api.example.com/users"
response = requests.get(url)
print(response.status_code)
print(response.text)
```

Reading JSON Response

If the server returns JSON → `data = response.json()`
`print(data)`

Query Parameters in Python

Don't manually construct URL. Use params:

```
→ params = {"city": "Bhopal", "page": 2}
  response = requests.get(url, params=params)
# requests will construct something like: ?city=Bhopal&page=2
```

Path Parameters

~~Python docs~~ → identifies resource `/users/25`.

Query → filters/modifies request `/users?page=2&limit=20`

JSON, Javascript Object Notation

common API data format.

JSON	Python
object →	dict
array →	list
string →	str
number →	int/float
true →	True
false →	False
null →	None

Python requests, used to make HTTP requests.

```
# import requests
response = requests.get(url)
data = response.json()
```

For request data,

```
requests.get(url, params=params)
        .post(" ", json=dat)
        .put(" ", " ")
        .patch(" ", " ").
        .delete(url)
```